



DS 3001

Midterm Project

Hudson, Jonathan, Tina, Sean, Sebastian

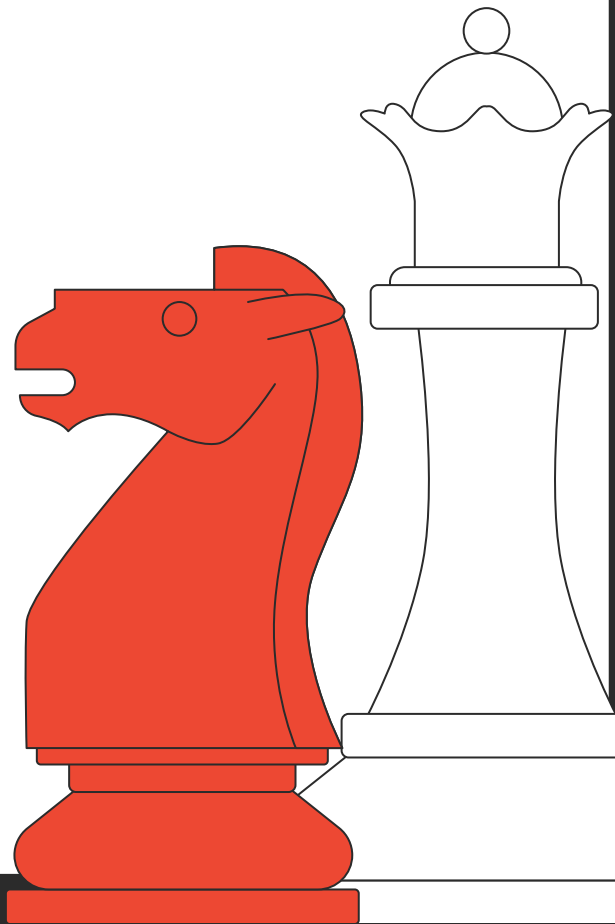
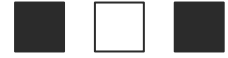


Table of contents



01

**Problem
Statement**

04

**Results and
Evaluation**

02

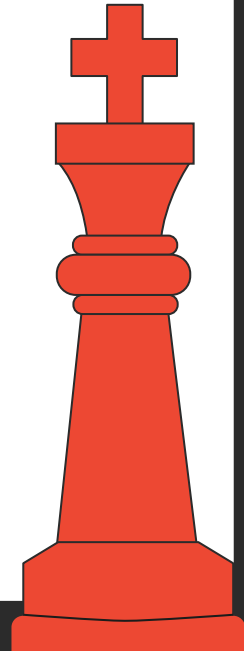
**Exploratory Data
Analysis**

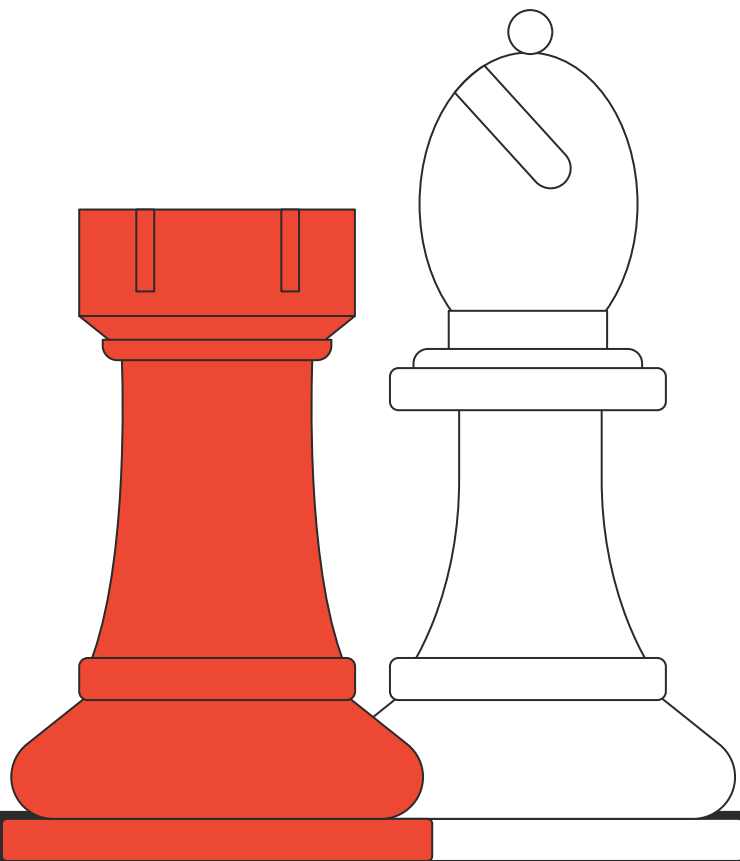
05

Other

03

Modeling





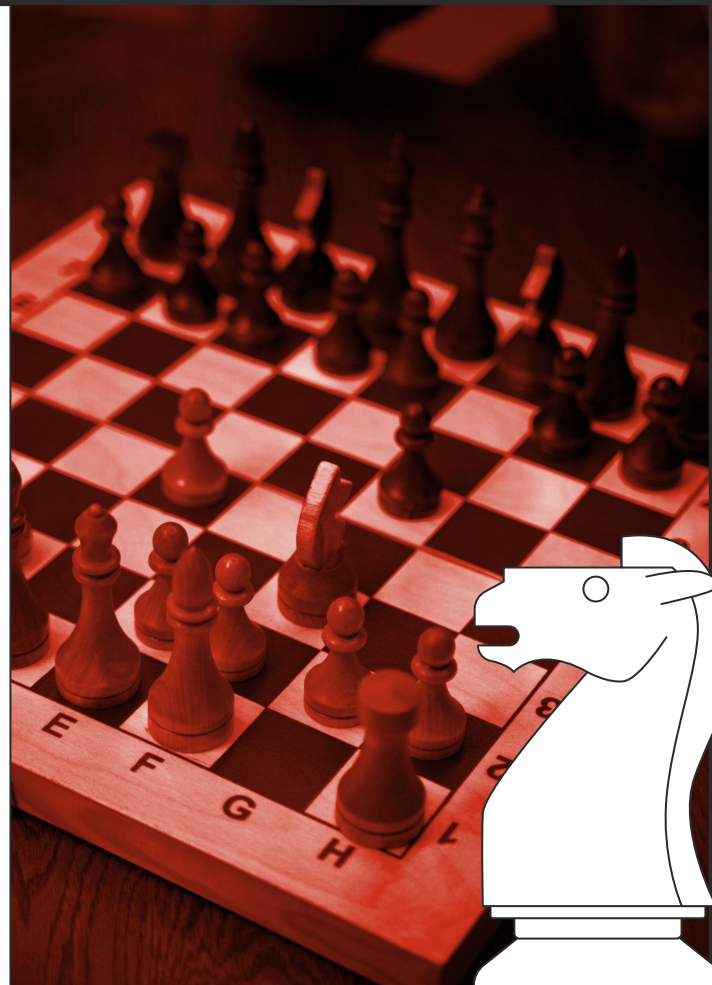
01 Problem Statement

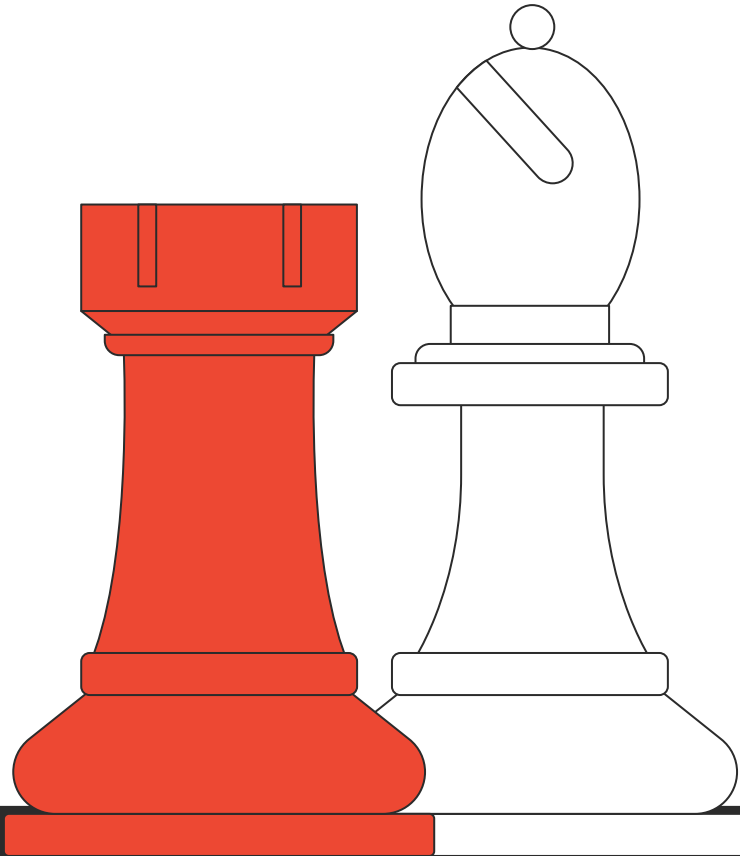
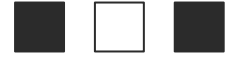


Problem Statement

- **Dataset:** Chess Game Dataset
- **Reason:** 20,000+ games, player ratings, opening choices, game length, well-structured data, etc
- **Project Objective:** Build a machine learning model to predict the winner, white or black, based on game factors.
- **Success Metrics:** Model accuracy, precision & recall, insights gained

kaggle





02 Exploratory Data Analysis

Exploratory Data Analysis



Dataset Structure

Size:

- 20,000+ games, structures in a tabular format

Key Variables:

- Numerical: Player ratings, number of moves, rating difference
- Categorical: Opening name, victory status, winner (white/black)
- Target Variable: winner_binary (Binary: White = 1, Black = 0)



Key Insights

Rating Difference Impact:

- Players with a **200+** point advantage win **~80%** of the time
- Average rating difference is **~8** points, most games are fair
- Some games have rating difference of **1500**, showing large skill mismatches

Opening Influence:

- Scandinavian Defense has the highest win rate for White (**~65%**)
- Van't Kruijs Opening has the lowest win rate for white (**~36%**), one could infer it's weak for White



Exploratory Data Analysis (cont.)



Key Insights

Longer Games:

- Queen's Gambit Declined: Hastings Variation averages **349** moves per game, longest-playing opening
- Neo-Gruenfeld and Semi-Slav Defense also lead to longer games (~**170-200** moves)



Justification for Modeling Approach

Strong Predictive Features:

- Rating difference, opening choice, and game length show clear correlations with match outcomes, making them useful for prediction

Clear Classification Problem:

- The goal is to predict a binary outcome (White/Black), making this ideal for supervised learning models

Feature Engineering:

- One-hot encoding categorical variables and creating numerical features optimize the dataset for machine learning

Machine Learning Feasibility:

- With many games and a distribution of winners, this dataset is large enough for training without major imbalance issues

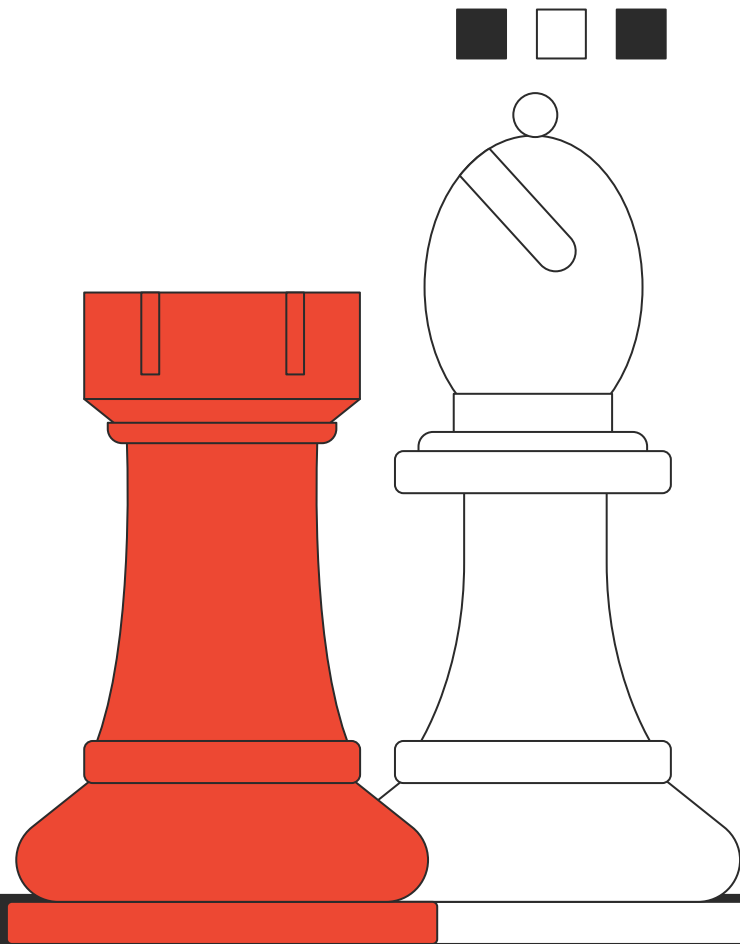
Real-World Applications:

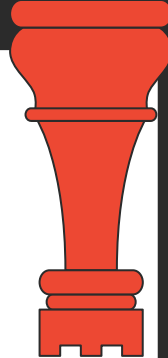
- Predicting chess outcomes based on pre-game factors can be useful for chess coaching, AI training, and competitive strategy analysis



03

Modeling





Logistic Regression

- **Purpose:** Acted as a baseline
- **Implementation:**
 - Used L1 regression to perform feature selection

Accuracy: 0.66

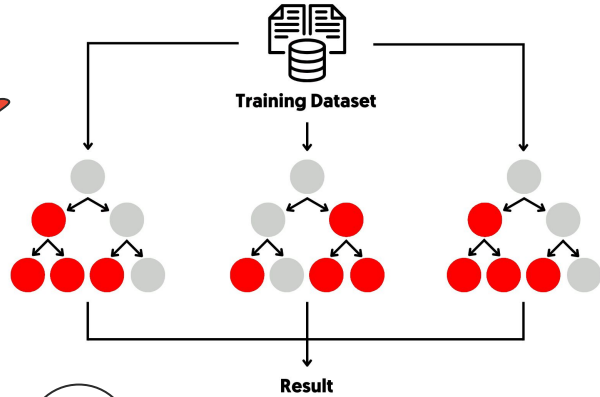
Classification Report:

	precision	recall	f1-score	support
0	0.65	0.60	0.62	1822
1	0.66	0.71	0.68	2000
accuracy			0.66	3822
macro avg	0.65	0.65	0.65	3822
weighted avg	0.66	0.66	0.65	3822

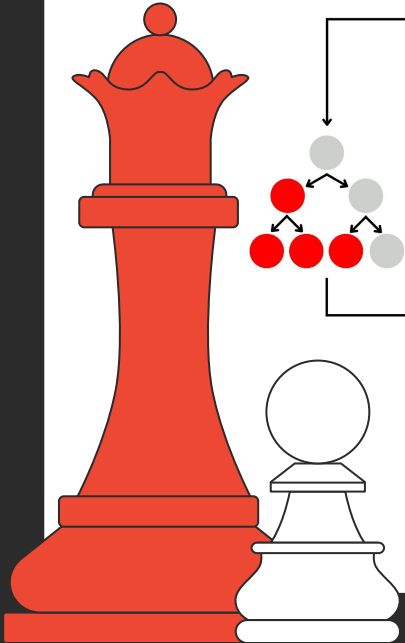




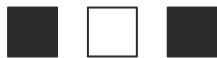
Random Forest



- **Purpose:** Captured non-linear relationships that logistic regression cannot handle
- **Implementation:**
 - Used 200 decision trees for better generalization
 - Looked at feature importance to see which factors influenced predictions the most



Modeling

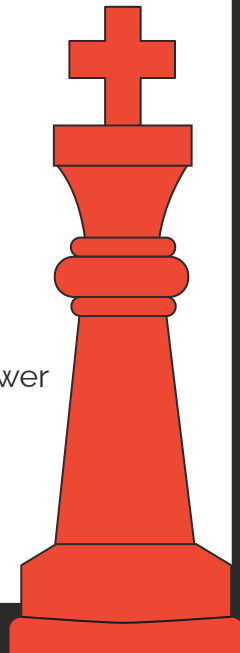


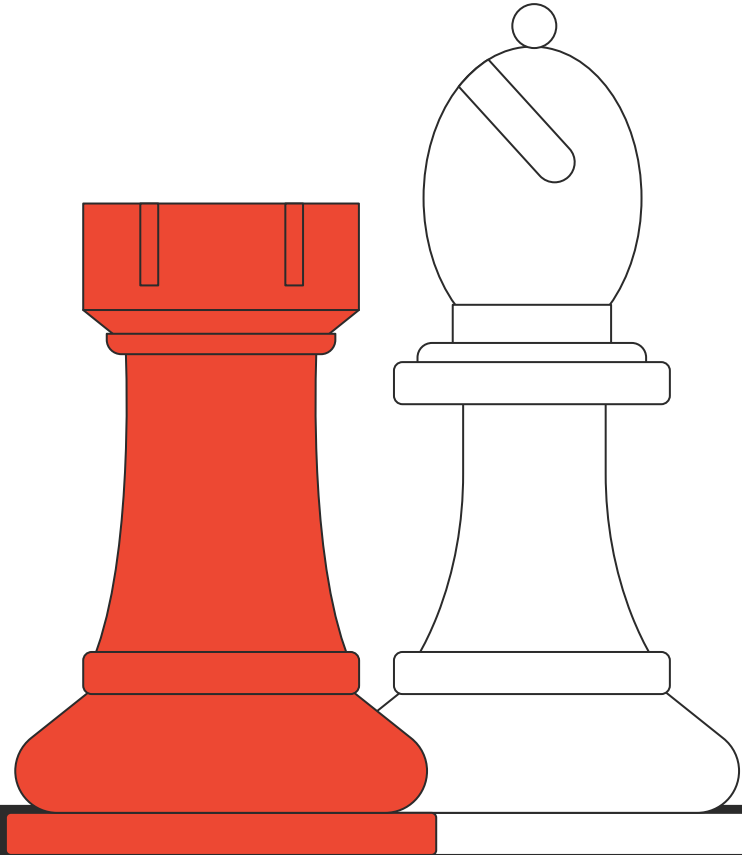
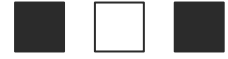
Data Processing

- Removed Draws, focusing on Black vs. White wins
- **Feature Engineering:**
 - **Rating differences** (white - black rating)
 - **Number of moves** in a game
 - **Top 10 most common openings** were one-hot encoded
 - **Victory status** also one-hot encoded
- **Train-test Split:**
 - **80%** training/**20%** testing
 - Preserve class distribution
- **Feature Scaling**
 - StandardScaler for logistic regression

Challenges

- **Logistic Regression's Limited Predictive Power:**
 - **Problem:** Chess outcomes involve complex patterns that linear models struggle with.
 - **Solution:** Used a random forest which has better nonlinear interactions
- **Feature Selection & Engineering:**
 - **Problem:** Some variables contributed little predictive power
 - **Solution:** Feature importance analysis guided the removal of weak features





04 Results and Evaluation

Performance Metrics

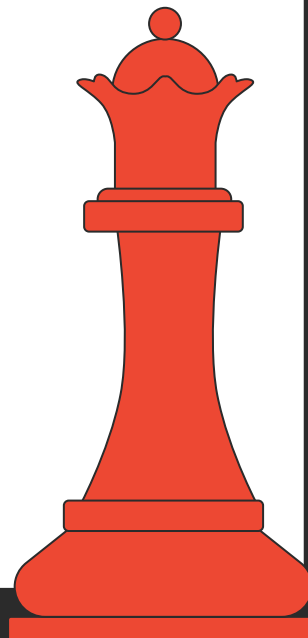


Model	Accuracy	Precision (White)	Recall (White)	F1-Score (White)	Precision (Black)	Recall (Black)	F1-Score (Black)
Logistic Regression	66%	66%	71%	68%	65%	60%	62%
Random Forest	71%	73%	72%	72%	70%	70%	70%



Observations

- **Outcome for Logistic Regression:** 66% - was better than random guessing
- **Outcome for Random Forest:** 71% - showed an improvement
- Random Forest performs better than logistic regression in all metrics
 - **Accuracy** improvement from **66% to 71%**
 - Random forests captures complex relationships better
 - **Precision of white wins** went from **66% to 73%**
 - Fewer false positives
 - **Recall of black wins** went from **60% to 70%**
 - Recovers more black win cases, less false negatives
 - Overall **balanced F1-scores** of **72% white** and **70% black**
 - Good generalizations



Visualizations



Confusion Matrix

- There were a lot more **false positives 732** (Was black but predicted white)
- Logistic Regression struggles with **complex decision boundaries**

Confusion Matrix:
[[1090 732]
[585 1415]]

Feature Importance (RF)

- The most influential feature was the **rating difference**
- Second was the **number of moves**

```
Feature Importance:

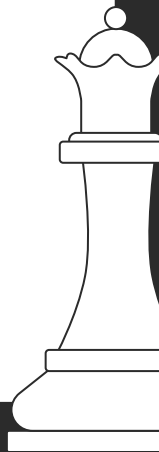
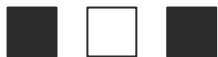
```

	Feature	Importance
0	rating_diff	0.496304
1	num_moves	0.473556
13	victory_status_resign	0.007049
12	victory_status_outoftime	0.004265
3	opening_simplified_Other	0.003315
...		
6	opening_simplified_Scandinavian Defense	0.001590
7	opening_simplified_Scandinavian Defense: Miese...	0.001573
5	opening_simplified_Queen's Pawn Game: Mason At...	0.001571
2	opening_simplified_Horwitz Defense	0.001469



What went well or bad

- **What went well:**
 - Changing from logistic regression to random forest went really well boosting our accuracy to 71%
 - During our feature engineering choosing rating difference and the number of moves helped improve accuracy
- **Challenges and limits:**
 - Our starting from logistic regression which had an accuracy of 66% was alright but there could of been better
 - Victory status and opening name had less of an effect than expected.
 - There could of been more features



Future Work and Enhancements

Try More Advanced Models:

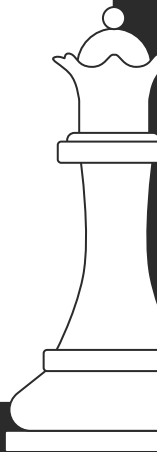
- Gradient Boosting (XGBoost, LightGBM, and CatBoost)
 - Comparing Gradient Boosting Algs
- Neural Networks (AlphaZero)
 - Understanding AlphaZero

Hyperparameter Tuning

- Optimize Random Forest parameters
 - Hyper-Parameter Sweep on AlphaZero
 - Ex: Adjusting # of trees, depth, and learning rate
 - Using GridSearchCV or Bayesian Optimization to find efficient hyperparameter combinations

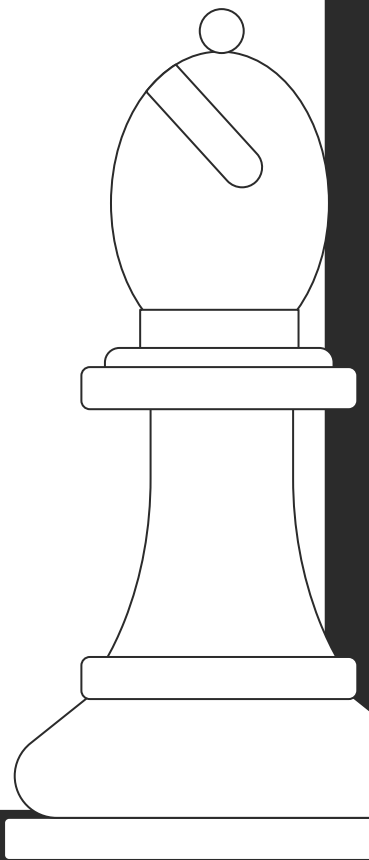
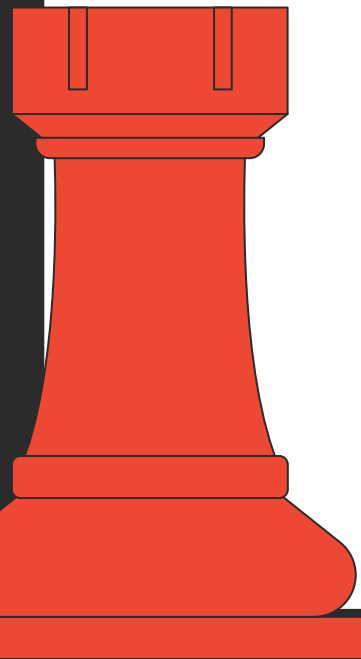
Creating a Chess Engine

- Instead of predicting who will win, we can train a model to play most optimally
 - Ex: Stockfish



■ □ ■
Thanks!

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)



Resources



- <https://www.kaggle.com/datasets/datasnaek/chess>
- <https://www.kaggle.com/code/bhavinmoriya/comparing-gradient-boosting-als-on-chess-data>
- https://www.chess.com/blog/the_real_greco/understanding-alphazero-a-basic-chess-neural-network
- <https://arxiv.org/abs/1903.08129>
- <https://www.geeksforgeeks.org/hyperparameter-optimization-based-on-bayesian-optimization/>
- <https://www.analyticsvidhya.com/blog/2021/06/tune-hyperparameters-with-gridsearchcv/4>
- <https://www.geeksforgeeks.org/ml-gradient-boosting/#>
- <https://www.geeksforgeeks.org/confusion-matrix-machine-learning/>
- ChatGPT 4 to help with some of the code